

✓ Programming for AI

Assignment 1

Full Name: Dias Zakir

Group: AAI-2606M

```
import os
import csv
import json
import zipfile
from pathlib import Path
from datetime import datetime
```

✓ Task 1: Working with Folders and Files

✓ Creating the project structure

```
os.mkdir("data")
```

```
os.mkdir("results")
```

✓ Working with CSV files

```
columns = ["first_name", "last_name", "group", "grades"]

rows = [
    ["Dias", "Zakir", "AAI-2606M", [95, 92, 85]],
    ["Bibarys", "Mukatai", "IS-2301", [87, 95, 90]],
    ["Daniyal", "Adilbekov", "SE-2411", [90, 85, 95]]
]

with open("data/students.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(columns)
    for row in rows:
        writer.writerow(row)
```

```
with open("students.csv", "r", encoding="utf-8") as f:
    print(f.read())
```

```

Snomanov Anei,AI-24-01,89,74,84,72,93
Abdullayev Karina,AI-24-02,81,78,73,73,95
Imanbekov Gulnaz,SE-22-01,80,97,83,80,76
Tleubayeva Aidosa,SE-22-02,75,86,83,86,68
Kenzhebay Damir,CS-23-01,92,77,84,64,95
Myrzakhmet Aknur,CS-23-02,70,66,81,81,62
Batyrbek Karina,CS-23-03,64,93,83,66,92
Tleubayeva Adil,AI-24-01,85,76,79,78,78
Akhmetova Aigerim,AI-24-02,68,82,86,65,61
Myrzakhmet Temirlan,SE-22-01,89,74,87,71,52
Turganbek Serik,SE-22-02,88,81,83,89,77
Zhaksylykov Nuray,CS-23-01,74,86,75,92,90
Kapanova Madi,CS-23-02,81,70,87,58,86
Batyrbek Madi,CS-23-03,88,75,84,88,96
Mansurova Nuray,AI-24-01,64,94,74,63,89
Tleubayeva Samat,AI-24-02,91,55,65,90,95
Omarov Rustem,SE-22-01,87,71,83,76,61
Imanbekov Arman,SE-22-02,84,80,94,83,90
Imanbekov Nuray,CS-23-01,80,82,70,84,72
Kydyr Miras,CS-23-02,62,95,86,85,63
Zhaksilik Almas,CS-23-03,72,57,73,59,75
Kenzhebay Zarina,AI-24-01,70,76,84,67,67
Imanbekov Aknur,AI-24-02,75,84,79,81,70
Kaldybek Ayaulym,SE-22-01,78,84,91,78,72
Beketov Damir,SE-22-02,61,93,96,72,94
Kaldybek Madi,CS-23-01,78,72,99,89,63
Zhussupov Gulnaz,CS-23-02,75,72,84,56,88
Kaldybek Gulnaz,CS-23-03,53,89,87,95,91

```

```

import csv

names = []
grades = []
average_grade = []

with open("students.csv", "r", encoding="utf-8") as f:
    reader = csv.reader(f)
    next(reader)
    for row in reader:
        names.append(row[0])
        grades.append([int(x) for x in row[2:]])

for name, grade in zip(names, grades):
    avg = sum(grade) / len(grade)
    average_grade.append(round(avg, 2))
    print(f"{name}: {grade} -> {avg:.2f}")

print(average_grade)

```

```

Zhaksylykov Nuray: [74, 86, 75, 92, 90] -> 83.40
Kapanova Madi: [81, 70, 87, 58, 86] -> 76.40
Batyrbek Madi: [88, 75, 84, 88, 96] -> 86.20
Mansurova Nuray: [64, 94, 74, 63, 89] -> 76.80
Tleubayeva Samat: [91, 55, 65, 90, 95] -> 79.20
Omarov Rustem: [87, 71, 83, 76, 61] -> 75.60
Imanbekov Arman: [84, 80, 94, 83, 90] -> 86.20
Imanbekov Nuray: [80, 82, 70, 84, 72] -> 77.60
Kydyr Miras: [62, 95, 86, 85, 63] -> 78.20
Zhaksilik Almas: [72, 57, 73, 59, 75] -> 67.20
Kenzhebay Zarina: [70, 76, 84, 67, 67] -> 72.80
Imanbekov Aknur: [75, 84, 79, 81, 70] -> 77.80
Kaldybek Ayaulym: [78, 84, 91, 78, 72] -> 80.60
Beketov Damir: [61, 93, 96, 72, 94] -> 83.20
Kaldybek Madi: [78, 72, 99, 89, 63] -> 80.20
Zhussupov Gulnaz: [75, 72, 84, 56, 88] -> 75.00
Kaldybek Gulnaz: [53, 89, 87, 95, 91] -> 83.00
[73.0, 68.8, 76.4, 72.8, 71.6, 73.4, 75.6, 85.4, 78.6, 76.4, 74.2, 79.0, 77.2, 82.0, 83.4, 77.8, 78.4, 76.8, 79.6, 77.8, 8

```

✓ Saving the results

```

import csv

with open("students.csv", "r", encoding="utf-8") as f:
    reader = csv.reader(f)
    columns = next(reader)
    rows = list(reader)

students = []

for row in rows:
    student = dict(zip(columns, row))
    subjects = columns[2:]
    student["grades"] = {s: int(student.pop(s)) for s in subjects}
    student["average_grade"] = round(
        sum(student["grades"].values()) / len(student["grades"]), 2
    )
    students.append(student)

print(students[0])

```

```
{'full_name': 'Shomanov Ayaulym', 'group': 'CS-23-01', 'grades': {'math': 56, 'physics': 71, 'cs': 69, 'history': 85, 'engli
```

```

write = True

if os.path.exists("results/report.json"):
    user_response = input("Warning! File exists. Do you want to rewrite? (Y/n): ")
    if user_response.lower() == "n":
        write = False
        print("Closing program...")
    elif user_response.lower() == "y":
        write = True
    else:
        write = False
        print("Not correct value...")

if write:
    with open("results/report.json", "w", encoding="utf-8") as f:
        json.dump(students, f, indent=4)
    print("Creating archive...")
    with zipfile.ZipFile("results.zip", "w") as zf:
        zf.write("results/")

```

```

Warning! File exists. Do you want to rewrite? (Y/n): Y
Creating archive...

```

```

with open("results/report.json", "r", encoding="utf-8") as f:
    data = json.load(f)

print(data)

```

```
{'full_name': 'Shomanov Ayaulym', 'group': 'CS-23-01', 'grades': {'math': 56, 'physics': 71, 'cs': 69, 'history': 85, 'engli
```

✓ Archiving

```

if write:
    with open("results/report.json", "w", encoding="utf-8") as f:

```

```
json.dump(students, f, indent=4)
print("Creating archive...")
with zipfile.ZipFile("results.zip", "w") as zf:
    zf.write("results/")
```

Creating archive...

Working with pathlib

```
p = Path("results/report.json")

print(p.exists())
print(p.stat().st_size)
print(datetime.fromtimestamp(p.stat().st_mtime))
```

True
40418
2026-09-26 13:40:27.657266

Task 2: Search for a Word or Phrase in Text Files

```
from genericpath import exists
texts = [
    "Python is a popular programming language.",
    "The weather in Astana is cold in winter.",
    "Machine learning uses Python and data.",
    "Go is used for backend development.",
    "Kazakhstan is a large country in Central Asia.",
]

os.makedirs("task2", exist_ok=True)

for i in range(5):
    with open(f"task2/name_{i+1}.txt", "w", newline="", encoding='utf-8') as f:
        f.write(texts[i])
```

```
found = []

user_query = input("Type value to search in documents: ")

for file in sorted(os.listdir("task2")):
    if os.path.splitext(file)[1] == ".txt":
        with open(os.path.join("task2", file), "r", encoding="utf-8") as f:
            if user_query in f.read():
                found.append(file)

if found:
    print(found)
else:
    print("Value not found in directory")
```

Type value to search in documents: Python
['name_1.txt', 'name_3.txt']

Task 3: File Information

```
with open("file.txt", encoding="utf-8") as f:
    text = f.read()

latin_letters = sum(1 for ch in text if ch.isascii() and ch.isalpha())
words = len(text.split())
lines = len(text.splitlines())

print("Latin letters:", latin_letters)
print("Words:", words)
print("Lines:", lines)
```

Latin letters: 236
Words: 52
Lines: 6

Task 4: Replacing Words

```

with open("words.txt", "r", encoding="utf-8") as f:
    text = f.read()

with open("forbidden_words.txt", "r", encoding="utf-8") as f:
    forbidden_words = f.read().split()

result = text

for word in forbidden_words:
    start = 0
    while True:
        index = result.lower().find(word.lower(), start)
        if index == -1:
            break
        result = result[:index] + "*" * len(word) + result[index + len(word):]
        start = index + len(word)

print(result)

```

***** is a high-level, interpreted *****ing **nguage that has gained immense popu**rity in recent years. It was first cr
 ***** is known for its simplicity and ease of use, which makes it an ideal **nguage for beginners. It has a straightforward

Answers to Self-Check Questions

1. How can you open a file for reading and writing?

Use `open()` with mode `'r+'`. It reads and writes without truncating, and the file must already exist.

```

with open("file.txt", "r+", encoding="utf-8") as f:
    data = f.read()
    f.write("new text")

```

The other read-write modes are `'w+'`, which truncates or creates the file, and `'a+'`, which appends and can also read.

2. Difference between `'r'`, `'w'`, `'a'`, `'rb'`, `'wb'`

Mode	Meaning
<code>'r'</code>	Read text (default). Raises <code>FileNotFoundError</code> if the file is missing
<code>'w'</code>	Write text. Creates the file or erases existing content
<code>'a'</code>	Append text to the end. Creates the file if missing
<code>'rb'</code>	Read binary. Returns <code>bytes</code>
<code>'wb'</code>	Write binary. Creates the file or erases it, and accepts <code>bytes</code>

3. Advantage of `with open(...) as f:`

It is a context manager. The file closes automatically when the block ends, even if an exception is raised. This avoids resource leaks and the need to call `close()` manually.

4. How to close a file correctly, and why

Call `f.close()`, ideally in a `try/finally` block, or use `with`, which does this for you. Closing matters for three reasons:

- It flushes buffered data to disk. Without it, written data may be lost.
- It releases the OS file descriptor, of which there is a limited number.
- It unlocks the file for other processes, which is especially important on Windows.

5. Main `os` functions for directories

- `os.getcwd()` returns the current directory, and `os.chdir()` changes it.
- `os.listdir()` and `os.scandir()` list a directory's contents.
- `os.mkdir()` and `os.makedirs()` create directories.
- `os.rmdir()` and `os.removedirs()` remove empty directories.
- `os.rename()` renames a file or directory.
- `os.walk()` traverses a directory tree recursively.

6. Creating and deleting folders

```

import os, shutil

os.mkdir("new_folder")          # single folder
os.makedirs("a/b/c", exist_ok=True)  # nested folders

```

```
os.rmdir("new_folder")          # only if empty
shutil.rmtree("a")              # folder with all contents
```

7. `os.getcwd()` vs `os.chdir()`

- `os.getcwd()` **returns** the path of the current working directory.
- `os.chdir(path)` **changes** the current working directory to `path`.

8. Listing files and folders

```
import os
from pathlib import Path

os.listdir(".")                # list of names
[e.name for e in os.scandir(".")] # entries with metadata
list(Path(".").iterdir())       # Path objects
list(Path(".").glob("*.txt"))   # by pattern
```

9. What `os.path` is for

It handles path manipulation and checks:

- Joining paths: `join`
- Splitting paths: `split`, `splittext`, `basename`, `dirname`
- Checking existence and type: `exists`, `isfile`, `isdir`
- Converting to absolute paths: `abspath`
- Getting file size: `getsize`

10. Checking whether a file or folder exists

```
import os
from pathlib import Path

os.path.exists("file.txt")    # file or folder
os.path.isfile("file.txt")
os.path.isdir("folder")

Path("file.txt").exists()
Path("folder").is_dir()
```

11. Why `pathlib` is more convenient than `os.path`

- It is object-oriented: a path is a `Path` object with methods, not a plain string.
- The `/` operator joins paths: `Path("data") / "file.txt"`.
- It has built-in helpers such as `.read_text()`, `.write_text()`, `.exists()`, `.mkdir()`, `.glob()`, `.suffix`, `.stem` and `.parent`.
- It is cross-platform by design, and the code is shorter and more readable.

12. Cross-platform path joining

```
import os
from pathlib import Path

os.path.join("data", "files", "a.txt")
Path("data") / "files" / "a.txt"
```

Both use the correct separator for the OS: `\` on Windows and `/` on Linux and macOS.

13. Methods for reading a CSV file

```
import csv

with open("data.csv", newline="", encoding="utf-8") as f:
    for row in csv.reader(f):          # rows as lists
        print(row)

with open("data.csv", newline="", encoding="utf-8") as f:
```

```
for row in csv.DictReader(f):      # rows as dicts (header = keys)
    print(row["name"])
```

Another option is `pandas.read_csv("data.csv")`, which loads the file into a DataFrame.

14. Advantage of the `csv` module over plain line reading

A naive `line.split(",")` fails in several common cases. The `csv` module handles them correctly:

- Quoted fields containing commas, such as `"Astana, KZ"`
- Newlines inside quoted fields
- Escaped quotes
- Custom delimiters and dialects, such as `;` or `\t`

It also maps columns to header names through `DictReader` and `DictWriter`.

15. Serializing Python data to JSON

```
import json

data = {"name": "Dias", "skills": ["Go", "Python"]}

s = json.dumps(data, ensure_ascii=False, indent=2)  # to string
with open("data.json", "w", encoding="utf-8") as f:
    json.dump(data, f, ensure_ascii=False, indent=2) # to file

obj = json.loads(s)                                # from string
with open("data.json", encoding="utf-8") as f:
    obj = json.load(f)                              # from file
```

16. JSON vs CSV

	JSON	CSV
Structure	Hierarchical: nested objects and lists	Flat table of rows and columns
Types	Strings, numbers, booleans, null	Everything is a string
Size	More verbose	Compact
Typical use	APIs, configs, complex data	Tabular data, Excel, data exports

17. Python types that map directly to JSON

Python	JSON
<code>dict</code>	object
<code>list</code> , <code>tuple</code>	array
<code>str</code>	string
<code>int</code> , <code>float</code>	number
<code>True</code> / <code>False</code>	true / false
<code>None</code>	null

Types such as `set`, `datetime`, `bytes` and custom objects need conversion first, for example through `default=`. Dict keys become strings.

18. Lazy loading

Lazy loading means data is loaded or computed **only when it is actually needed**, not all at once. This has three benefits:

- Memory use stays low, so files larger than RAM can be processed.
- Processing can start sooner.
- No work is wasted on data that never gets used.

19. Reading a large file line by line

```
with open("big.txt", encoding="utf-8") as f:
    for line in f:      # reads one line at a time
        process(line)
```

For binary data, read in chunks instead:

```
with open("big.bin", "rb") as f:
    while chunk := f.read(8192):
        process(chunk)
```

20. `readline()` and iterating over a file object

- `f.readline()` reads **one line** per call and returns `""` at end of file. It is useful for reading a header or for controlled manual reading.
- `for line in f:` is the file's built-in lazy iterator. It is the idiomatic, memory-efficient way to process every line.
- For contrast, `f.readlines()` loads **all** lines into a list, so it is unsuitable for large files.

21. When to use generators with files

Use generators when:

- the file is large and should be processed as a stream;
- you are building a processing pipeline, such as read, then filter, then transform;
- you need only part of the data, such as the first N matches.

```
def read_errors(path):
    with open(path, encoding="utf-8") as f:
        for line in f:
            if "ERROR" in line:
                yield line.strip()

for err in read_errors("app.log"):
    print(err)
```

22. Modules for archives

- `zipfile` handles `.zip`.
- `tarfile` handles `.tar`, `.tar.gz` and `.tar.bz2`.
- `shutil` offers high-level helpers: `make_archive()` and `unpack_archive()`.
- `gzip`, `bz2` and `lzma` compress single files.

23. Creating a ZIP archive

```
import zipfile, shutil

with zipfile.ZipFile("archive.zip", "w", zipfile.ZIP_DEFLATED) as z:
    z.write("file1.txt")
    z.write("data/file2.csv", arcname="file2.csv")

shutil.make_archive("backup", "zip", "folder_to_archive") # whole folder
```

24. Extracting an archive to a specified directory

```
import zipfile, tarfile, shutil

with zipfile.ZipFile("archive.zip") as z:
    z.extractall("output_dir")

with tarfile.open("archive.tar.gz") as t:
    t.extractall("output_dir", filter="data") # safe filter (Python 3.12+)

shutil.unpack_archive("archive.zip", "output_dir")
```

Be careful with archives from untrusted sources, because they can contain paths like `../` that write outside the target directory.

25. Key considerations for binary files

- Open them in binary mode (`'rb'` or `'wb'`). Otherwise Python decodes or encodes the content and corrupts it.
- Don't pass `encoding=`. You work with `bytes`, not `str`.
- Read large files in chunks rather than all at once.
- Byte structure matters. Use `struct` to parse formats and check magic numbers such as the PNG signature `b'\x89PNG'`.
- Use `seek()` and `tell()` for random access.
- Copy files with `shutil.copyfile()` instead of reading and writing them manually.

Напишите программный код или [сгенерируйте](#) его с помощью искусственного интеллекта.

